



Malware in Disguise: Crafting Adversarial Executables

Danny Kim

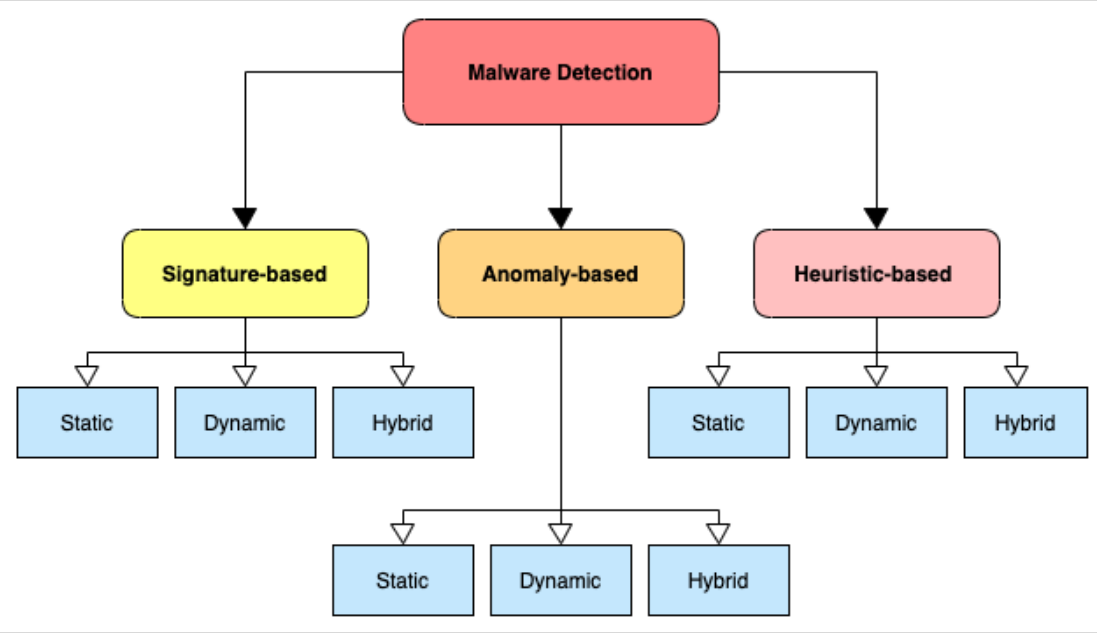
Introduction

Project Description

This project focuses on static analysis as that is the more popular form of malware detection. This project displays random section injection/padding attacks against the baseline EMBER detector model.

Problem Context

Traditional malware detection methods, such as signature-based detection, rely on predefined patterns identified by security professionals and are effective for known malware but struggle against new or unknown threats. Due to these limitations, Machine Learning (ML)--based malware detectors have gained popularity.



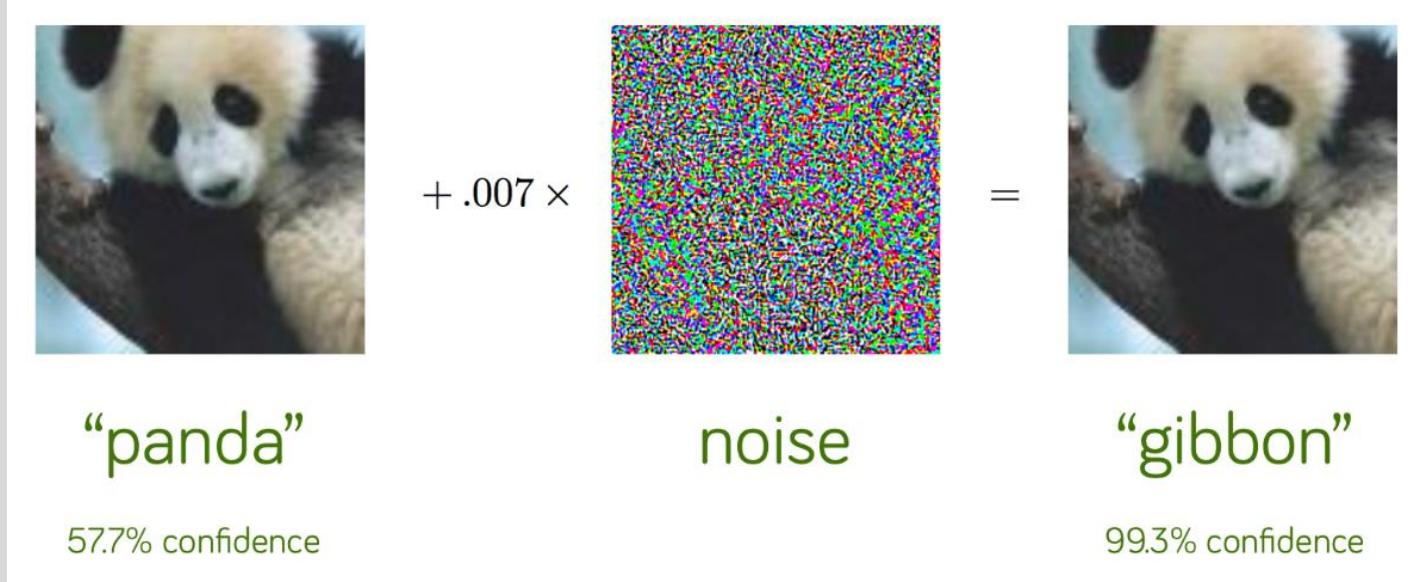
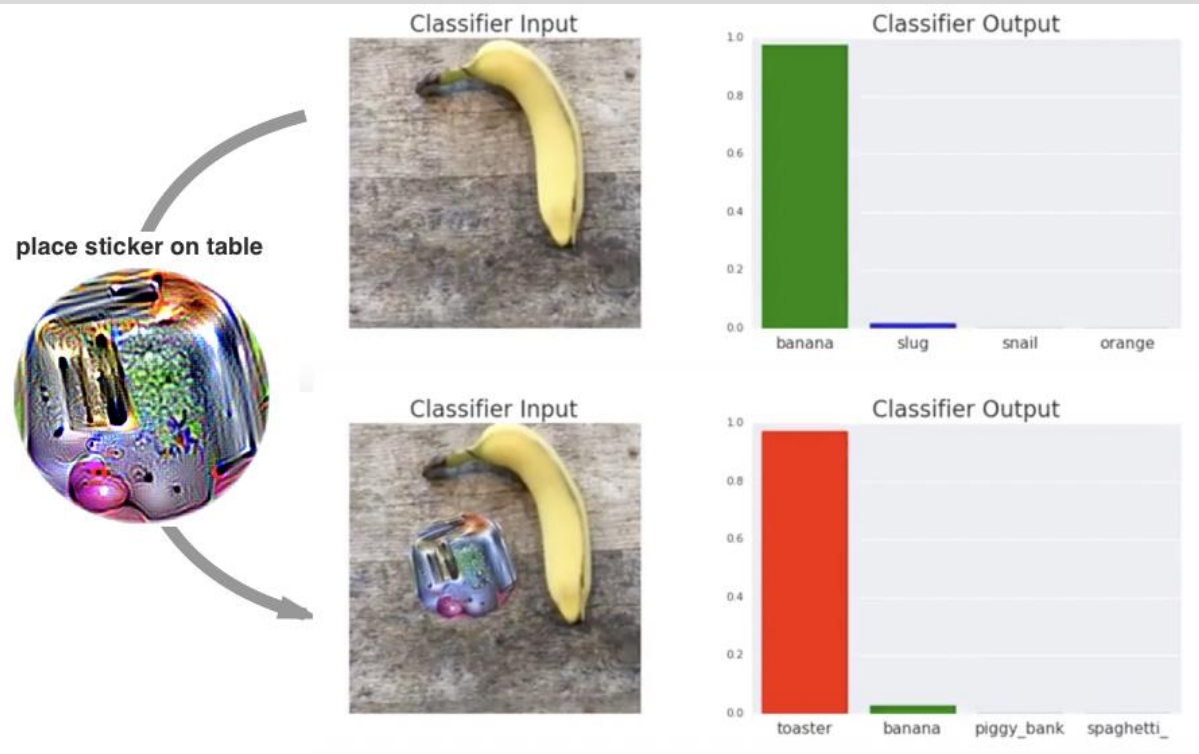
J. Hossain, Hossain Shahrir, M. Valero, Farhat Lantia Barsha, and F. Wu, "Malware Detection and Prevention using Artificial Intelligence Techniques," Jun. 25, 2022.
https://www.researchgate.net/publication/361581211_Malware_Detection_and_Prevention_using_Artificial_Intelligence_Techniques

Takeaway

Although ML-based malware detectors provide multiple benefits over traditional malware detectors, ML-based detectors are vulnerable to adversarial attacks. . As ML based malware detectors are soon being integrated into being the standard in the cybersecurity industry, organizations have to be wary of adversarial attacks

Background

What are Adversarial Attacks?

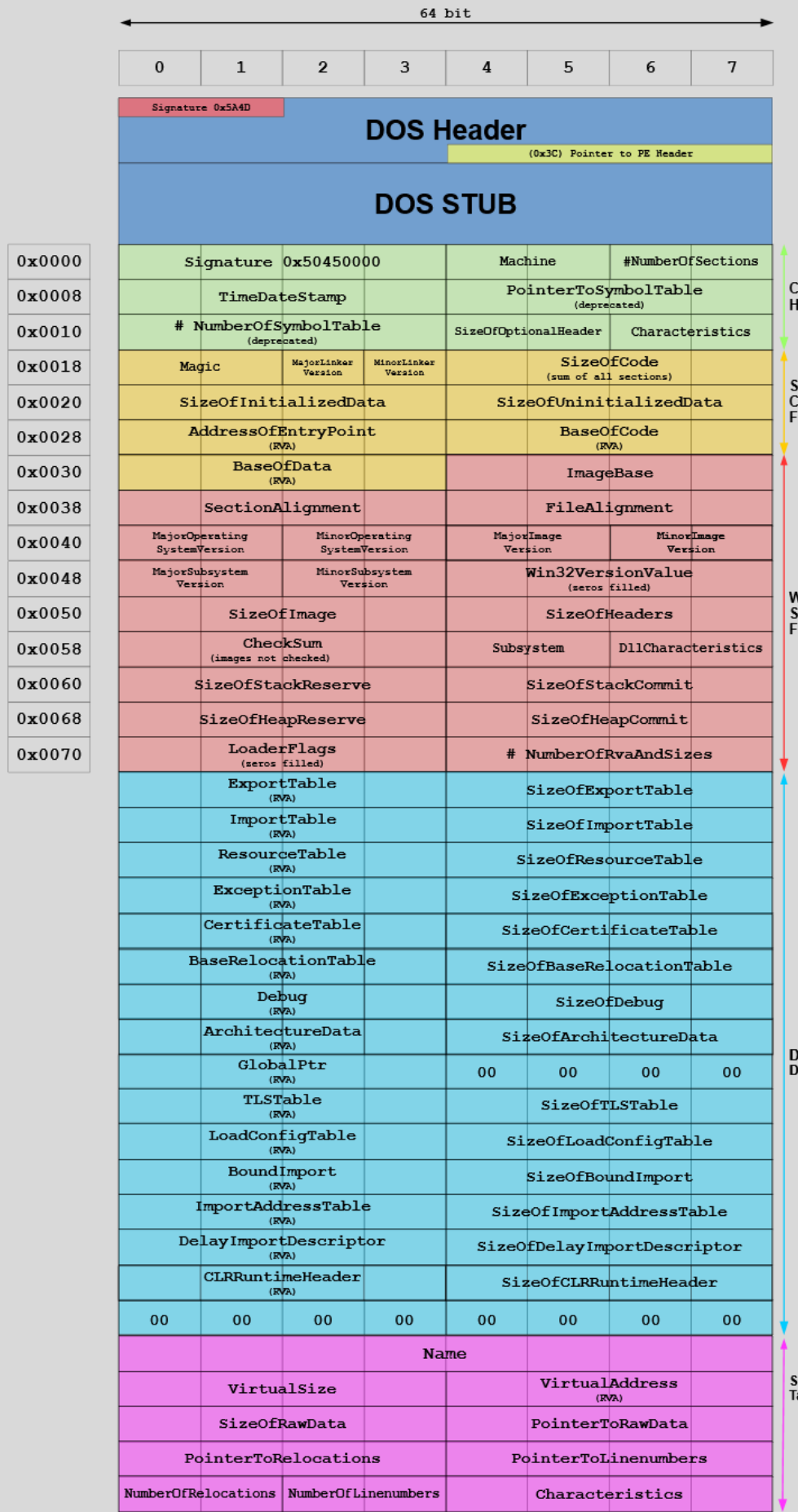


"How to confuse antimlware neural networks. Adversarial attacks and protection," securelist.com. <https://securelist.com/how-to-confuse-antimlware-neural-networks-adversarial-attacks-and-protection/102949/>

Adversarial attacks in the context of this project refer to evasion tactics utilized by an adversary. Evasion attacks describe a type of attack where the adversary deliberately manipulates the input example with different levels of perturbations, intending to fool the ML model into misclassifying the input

PE File Format

- DOS Header**
provides metadata needed for loading the program in a DOS environment.
- DOS Stub**
a tiny program embedded for backward compatibility. When run in an MS-DOS environment, it prints the message "This program cannot be run in MS-DOS mode."
- Optional Header**
Provides information needed by the loader, including the Magic value (PE32 vs PE32+), the program entry point, required memory sizes, and other parameters
- PE Sections**
PE sections store the executable's actual contents: code, data, and resources.

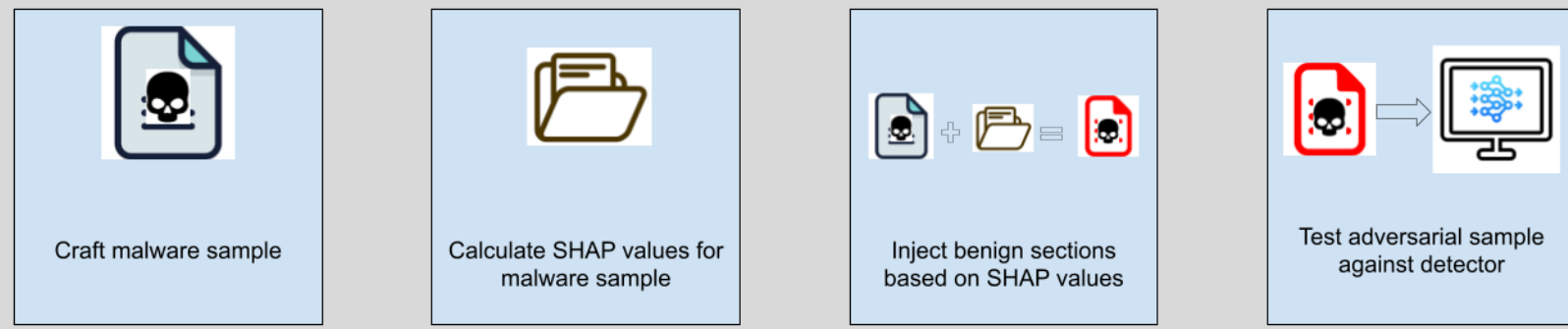


Wikipedia
https://upload.wikimedia.org/wikipedia/commons/1/1b/Portable_Executable_32_bit_Structure_in_SVC_fixed.svg

Approach

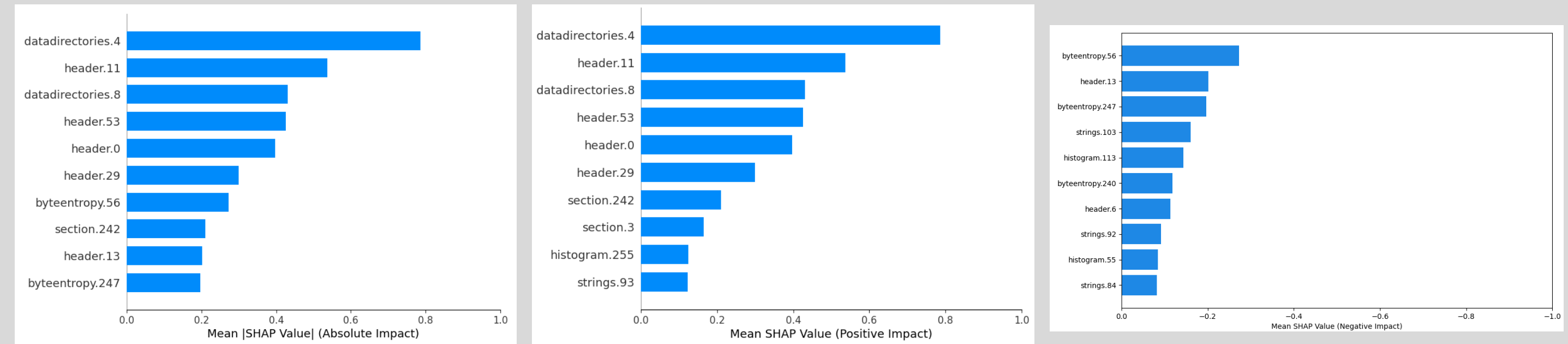
Adversarial Sample Generation:

1. Craft malware sample
2. Calculate the SHAP values for the malware sample
3. Craft an adversarial malware sample using the SHAP values
4. Test the adversarial malware sample against the EMBER detector

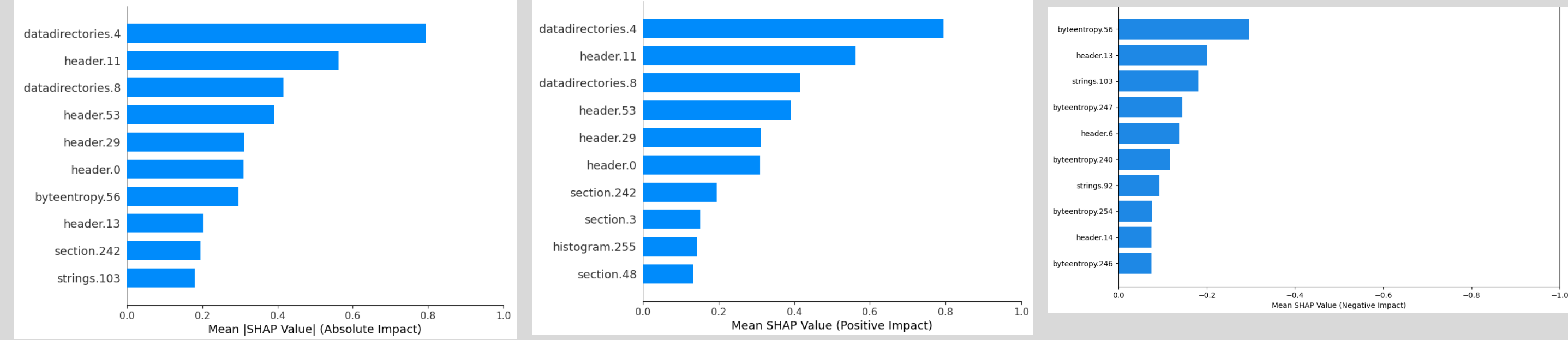


Results

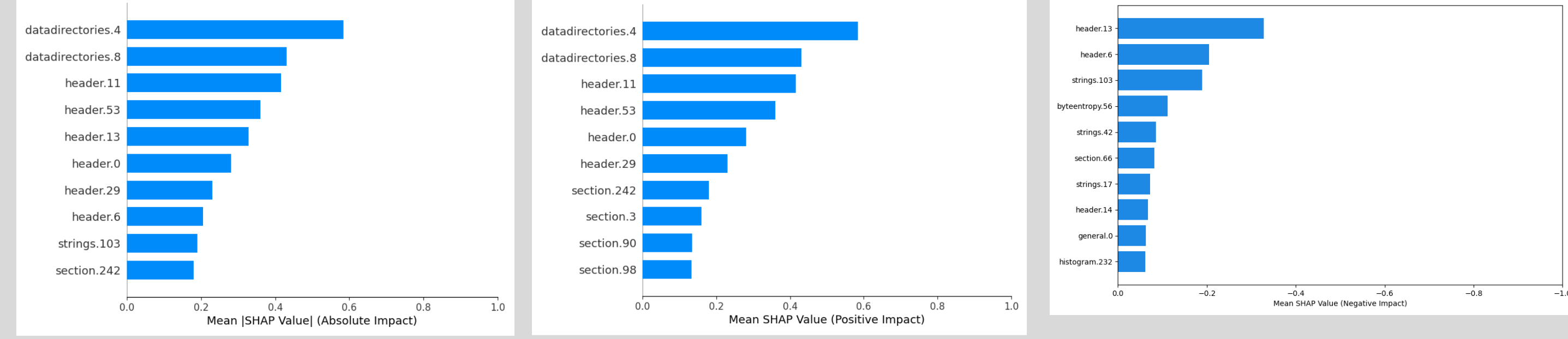
SHAP Values of Original Sample



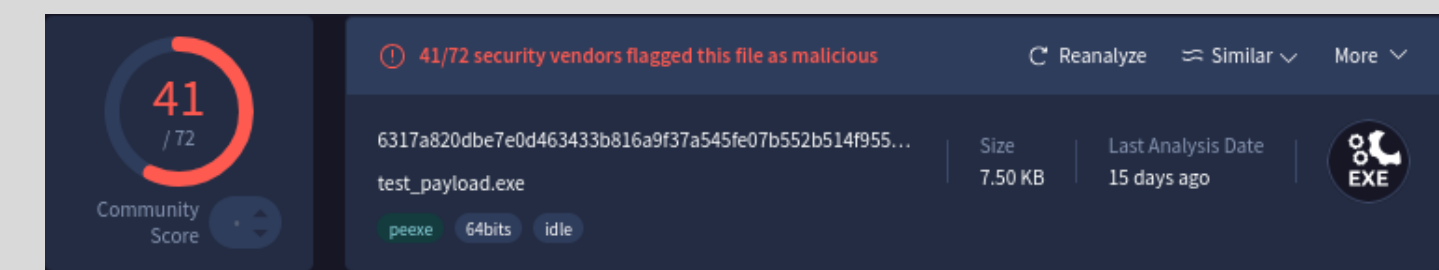
SHAP Values of Modified Sample (1 Added Section)



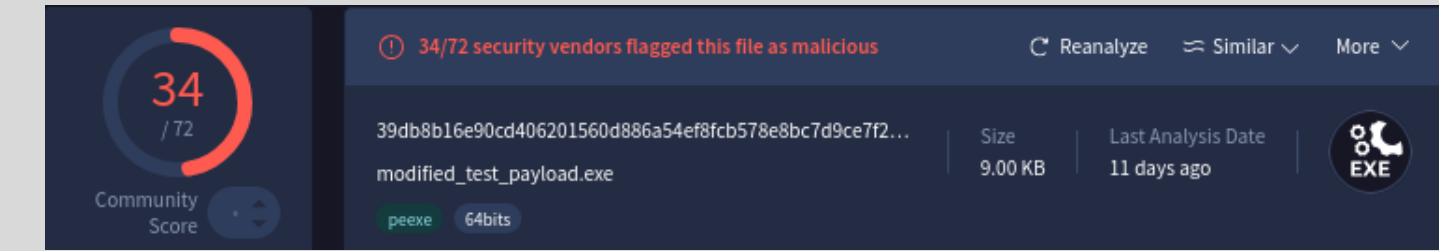
SHAP Values of Modified Sample (6 Added Sections)



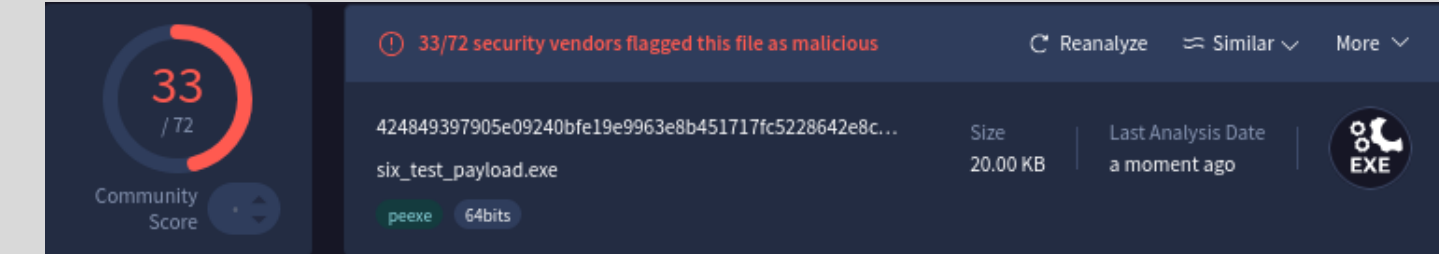
Original Sample against VirusTotal



Modified Sample (1 Added Section) against VirusTotal



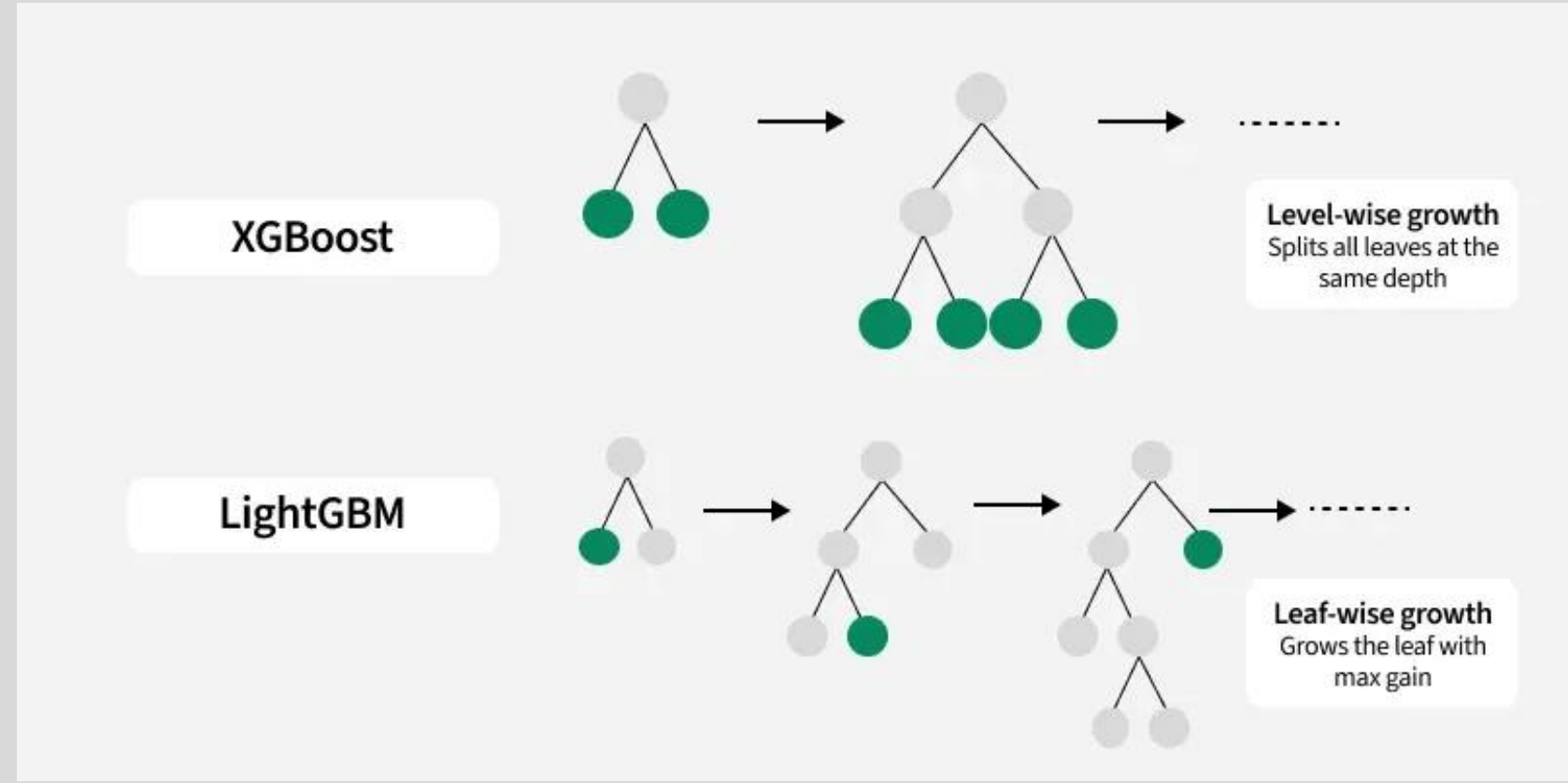
Modified Sample (6 Added Sections) against VirusTotal



FAQ

What is EMBER based off?

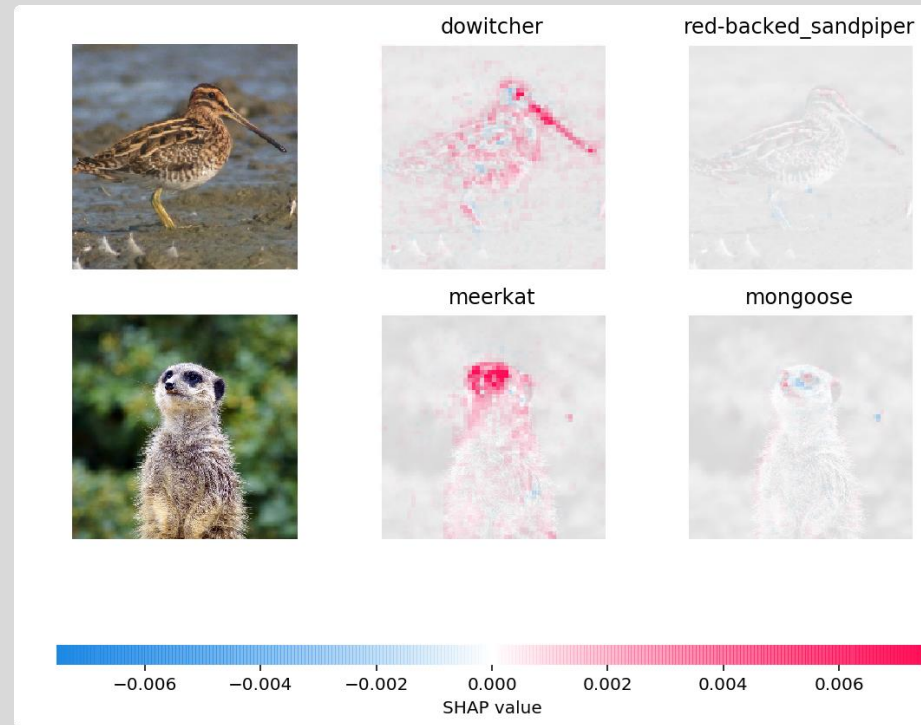
The detector used in this project is the baseline EMBER model, a Light Gradient Boosted Decision Tree (LightGBM). Gradient boosted decision trees combine many weak learners, small decision trees, into a single strong classifier. Each tree is trained to reduce the residual error left by the previous trees, allowing the model to iteratively improve its predictions.



GeeksforGeeks, "LightGBM (Light Gradient Boosting Machine)," GeeksforGeeks, Jul. 15, 2020.
<https://www.geeksforgeeks.org/machine-learning/lightgbm-light-gradient-boosting-machine/>

LightGBM differs from traditional gradient boosting methods in how it constructs trees. Instead of growing trees level-by-level, it grows them leaf-wise, always splitting the leaf that provides the largest reduction in loss. This produces deeper, more expressive trees and generally leads to better performance on large, high-dimensional datasets like EMBER.

What is SHAP?



"How to confuse antimlware neural networks. Adversarial attacks and protection," securelist.com. <https://securelist.com/how-to-confuse-antimlware-neural-networks-adversarial-attacks-and-protection/102949/>

SHAP values (SHapley Additive exPlanations) measure how much each feature contributes to a model's prediction. A positive SHAP value means the feature pushed the prediction toward "malicious," while a negative value pushed it toward "benign."

How Does the EMBER detector work?

1. Extract a fixed set of static PE features from the input sample.
2. Convert these features into a structured numerical vector.
3. Pass this vector into the LightGBM model, which outputs a malware/benign prediction.

Which Features are Extracted?

Parsed PE Features:

- General File Info: Basic attributes such as file size, virtual size, number of imports/exports, and whether the file has resources, TLS, relocations, or a digital signature.
- Header Information: Metadata from the COFF and Optional headers: machine type, subsystem, DLL characteristics, version numbers, and memory layout information.
- Imported Functions: List of DLLs and API calls the binary depends on, extracted from the Import Address Table.
- Exported Functions: Any functions the binary exposes for other programs to call.
- Section Information: Per-section properties such as section names (.text, .data, .rdata), raw size, virtual size, entropy, and permission flags.

Format-Agnostic Features:

- Byte Histogram: Counts of each byte value (0–255) across the file, giving a coarse view of content distribution.
- Byte-Entropy Histogram: A 2D histogram capturing how byte values correlate with local entropy, useful for detecting packed or obfuscated regions.
- String Information: Statistics about printable strings (counts, average length, character distributions) and indicators such as URLs, file paths, or registry keys.